Домашнє завдання №1
Сортування і структури даних

1 Допустиме використання ШІ

Дозволяється використовувати ШІ для:

1) Реалізації одного підходу до вирішення задачі I;
2) Реалазації методу лінійної регресії;
3) Відображення красивих графіків;
4) Написання автоматичних тестів.

Помилки ШІ зменшують загальну оцінку

2 Постановка задачі

Вам необхідно реалізувати структуру даних, яка буде зберігати базу студентів у пам'яті і виконувати задані вашим варіантом операції 1, 2, 3 оптимальним чином, щоб досягти найбільшої кількості операцій, виконаних за 10 секунд.

Співвідношення кількості операцій 1, 2, 3 визначається вашим варіантом як $A : B : C$. Операції мають надходити у випадковому порядку і використовувати існуючі значення полів там, де це можливо. Операції необхідно підготувати наперед, щоб вони не впливали на повторення експериментів.

Ваша структура даних може суміщати різні контейнери для розв'язання задачі оптимальним чином. Наприклад: хеш-таблиця за полем `m_group`, BST за полем `m_surname`, посортований статичний масив за полем `m_birth_year`.

3 Структура бази даних

База даних містить такі поля. Її необхідно зчитати в пам'ять з файлу `students.csv`.

```
struct Student
{
    std::string m_name;          // Name in Ukrainian
    std::string m_surname;       // Surname in Ukrainian
    std::string m_email;         // [a-z.]*@student.org  is unique

    int m_birth_year;            // 1950..2010
    int m_birth_month;           // 1..12
    int m_birth_day;             // 1..Number of days in m_birth_month

    std::string m_group;         // [A-Z][A-Z][A-Z]-[0-9][0-9]
    float m_rating;              // 0..100

    std::string m_phone_number;  // 38(0xx)xx-xx-xxx is unique
};
```

4 Задача I - ініціалізація і виконання операцій

Програма має підтримувати такі операції:

- **Ініціалізація** — створення бази в пам'яті з непосортованого файлу на диску у форматі CSV (`students.csv`).
- **Операції 1, 2, 3** — визначаються варіантом V.

Варіант V1

1. Знайти студентів за заданим ім'ям і прізвищем (`m_name`, `m_surname`).
2. Знайти такі різні пари груп, де є студенти з однаковими (`m_name`, `m_surname`). Результат - список пар груп і будь-який один повторюваний (`m_name`, `m_surname`) до кожної з них.
3. Змінити групу студенту за його електронною поштою (`m_email`).

Варіант V2

1. Знайти всіх студентів за заданим днем народження (`m_birth_month`, `m_birth_day`).
2. Змінити групу студенту за його електронною поштою (`m_email`).
3. Для кожної групи порахувати кількість унікальних пар (`m_birth_month`, `m_birth_day`) серед студентів цієї групи

Варіант V3

1. Змінити групу студенту за його номером телефону (`m_phone_number`).
2. Повернути студентів групи `m_group` в алфавітному порядку за прізвищем і ім'ям (`m_surname`, `m_name`).
3. Повернути список груп, де навчаються студенти з прізвищем `m_surname`.

Варіант V4

1. Знайти групу або групи, в яких найбільше студентів.
2. Змінити групу студенту за його електронною поштою (`m_email`).
3. Знайти групу або групи, в яких найбільший середній рейтинг студентів.

Варіант V5

1. Повернути 100 студентів з найвищим рейтингом.
2. Встановити рейтинг студенту за його електронною поштою (`m_email`).
3. Знайти групу або групи з найбільшим середнім рейтингом студентів.

5 Задача II - сортування

Операція сортування визначається варіантом S. Сортування або за зростанням або в алфавітному порядку. Сортування має враховувати українську абетку там, де це важливо. Порядок записів з однаковим значенням ключа не визначається. Час побудови всіх допоміжних структур даних входить у час сортування.

S1. Посортувати всіх студентів за `m_email` і зберегти в CSV-файл.

S2. Посортувати всіх студентів за `m_rating` і зберегти в CSV-файл.

S3. Посортувати всіх студентів за (`m_name`, `m_surname`) і зберегти в CSV-файл.

S4. Посортувати всіх студентів за (`m_surname`, `m_name`) і зберегти в CSV-файл.

S5. Посортувати всіх студентів за `m_group` і зберегти в CSV-файл.

S6. Посортувати всіх студентів за (`m_birth_month`, `m_birth_day`) і зберегти в CSV-файл.

S7. Посортувати всіх студентів за `m_phone_number` і зберегти в CSV-файл.

6 Очікуваний результат й експерименти до задачі I

- Програма мовою програмування **C++** або **Java**, яка виконує необхідні операції.
- Запропонуйте **3 різні варіанти** вирішення задачі I з використанням різних контейнерів обраної мови програмування. Для кожного запропонованого варіанту необхідно експериментально порахувати кількість виконаних операцій за **10 секунд**. Операції 1, 2, 3 повинні випадковим чином обиратися у співвідношенні **A** : **B** : **C**, яке задане вашим варіантом у додатковій таблиці.
- Для найкращого варіанта з попереднього пункту необхідно виконати експеримент для 10 різних розмірів початкової бази даних від 100 до 100000 (проміжні значення і кількість експериментів >= 5 на одну точку треба обрати самостійно). Також необхідно порахувати **об'єм використаної пам'яті** і **середній час виконання кожної з операцій 1,2,3**. Методику оцінки використаної пам'яті обрати та обґрунтувати самостійно.
- Оцінити за допомогою **лінійної регресії**, як зростає час виконання найповільнішої з операцій 1,2,3 зі зростанням кількості записів в базі даних. Оцінка має бути отримана у вигляді функції

$$T(n) = c_1 \cdot f_1(n) + c_2 \cdot f_2(n) + c_3 \cdot f_3(n),$$

де функції f_1, f_2, f_3 необхідно підібрати відповідно до теоретичної оцінки складності складових алгоритму (наприклад, $f_1 = n^2, f_2 = n \log n, f_3 = n$), а коефіцієнти c_1, c_2, c_3 отримати методом лінійної регресії. Отримані результати і функцію $T(n)$ необхідно **подати таблично** та **зобразити на графіку**, де для кожної експериментально отриманої точки треба відобразити **середнє і середньо-квадратичне відхилення**.

На захисті необхідно показати:

- що обраний вами варіант реалізації є раціональним;
- що всі операції виконуються правильно;
- що коректність зберігається після виконання операцій зміни даних;
- що експерименти виконано правильно;
- як зміниться вибір структур даних, якщо система має бути стійка до DoS атаки (базу і запити формує ненадійний користувач)?

7 Очікувані результати задачі II

Для задачі II необхідно зробити **дві реалізації** та порівняти час сортування. Час запису результату у файл при вимірюванні часу сортування **не враховується**.

Необхідно реалізувати:

1. Сортування з використанням стандартної функції сортування обраної мови програмування.
2. Власну реалізацію алгоритму сортування з використанням **Radix Sort**, **BST** чи іншого підходу, який може пришвидшити сортування.

Після реалізації необхідно експериментально порівняти час роботи обох підходів в залежності від кількості записів в базі даних.